Lecture 11
Proving program correctness

Armando Solar-Lezama

Constraint-based verification

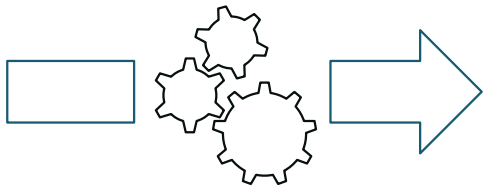
Code

```
if(x > y){
  t = x - y;
  while(t > 0){
    x = x - 1;
    y = y + 1;
    t = t - 1;
  }
  assert y=x_in
  assert x=y_in
}
```



Verification Condition Generation

$$\forall in \;.\; Q(in)$$



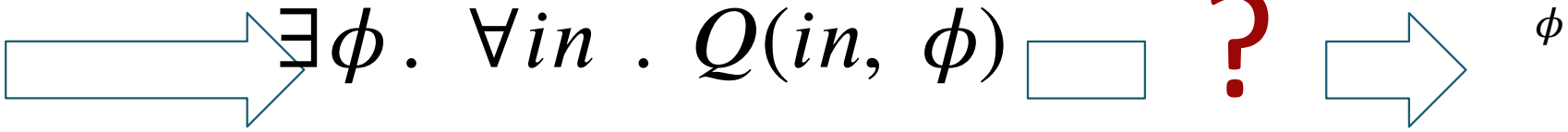
Solver

OK

… and Synthesis

Code

```
if(x > y){
  t = x - y;
  while(t > 0){
    x = x - ϕ(??1);
    y = y + ϕ(??2);
    t = t - ϕ(??3);
  }
  assert y=xin
  assert x=yin
}
```



Verification Condition Generation

Historically, solvers that can deal with quantifier alternations have not been very scalable

Example

Consider the following program

```
...
if(x > y){
  t = x – y;
  while(t > 0){
    x = x – 1;
    y = y + 1;
    t = t – 1;
  }
}
```

I claim that for any values of x and y

- the loop will terminate
- when it does, if x > y, the values of x and y will be swapped

How could I prove this?

Axiomatic Semantics

A system for proving properties about programs

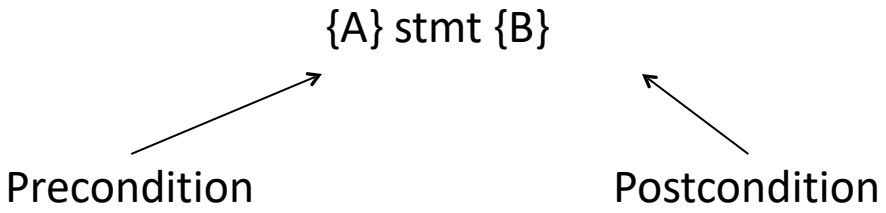
Key idea:

- we can define the semantics of a construct by describing its effect on assertions about the program state

Two components

- A language for stating assertions
  - can be First Order Logic (FOL) or a specialized logic such as separation logic.
  - many specialized languages developed over the years
    - Z, Larch, JML, Spec#
- Deductive rules for establishing the truth of such assertions

The basics



Hoare triple

- If the precondition holds before stmt and stmt terminates postcondition will hold afterwards

This is a partial correctness assertion

- we sometimes use the notation

[A] stmt [B]

to denote a total correctness assertion

- that means you also have to prove termination

What do assertions mean?

We first need to introduce a language

For today we will be using Winskel's IMP

e:= n | x | e_1 + e_2

c:= x := e | c_1 ; c_2 | if e then c_1 else c_2 | while e do c

Big Step Semantics have two kinds of judgments

$\langle e, \sigma \rangle \to n$ $\langle c, \sigma \rangle \to \sigma'$

expressions result in values commands change the state

What do assertions mean?

The language of assertions

- A := true | false | e1 = e2 | e1 >= e2 | A1 and A2 |
  not A | $\forall$ x . A

Notation $\sigma \vDash A$ means that the assertion holds on state σ

- This is defined inductively over the structure of A.
- Ex. $\sigma \vDash A \; and \; B \quad iff \quad \sigma \vDash A \; and \; \sigma \vDash B$

Partial Correctness can then be defined in terms of OS

{A} c {B} iff $\forall\sigma\forall\sigma'(\sigma \vDash A \wedge \langle c,\sigma\rangle \rightarrow \sigma') \Rightarrow \sigma' \vDash B$

Derivation Rules

Derivation rules for each language construct

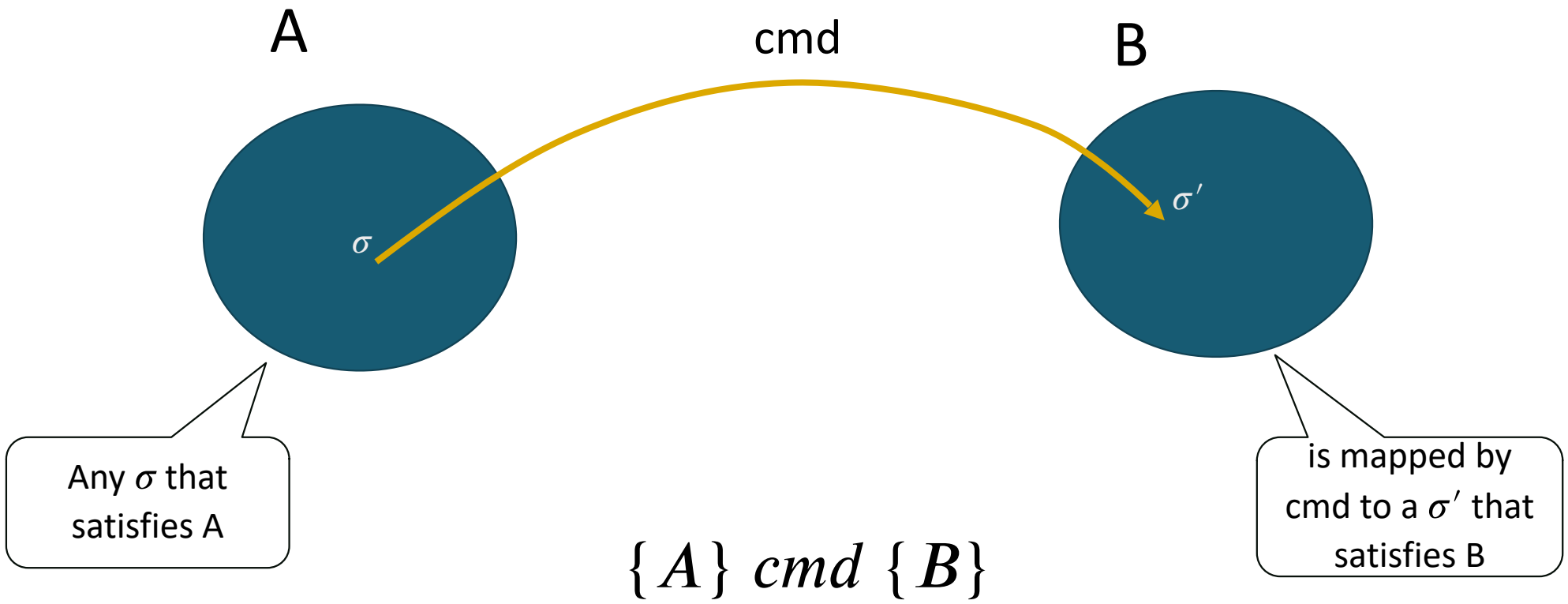
$$\frac{}{\vdash \{A[x \to e]\}x := e\,\{A\}} \qquad \frac{\vdash \{A \wedge b\}c_1\,\{B\} \quad \vdash \{A \wedge not\ b\}c_2\,\{B\}}{\vdash \{A\}if\ b\ then\ c_1 else\ c_2\,\{B\}}$$

$$\frac{}{\vdash \{A\}skip\{A\}} \qquad \frac{\vdash \{A\}c_1\,\{C\} \quad \vdash \{C\}c_2\,\{B\}}{\vdash \{A\}c_1; c_2\,\{B\}} \qquad \frac{\vdash \{A \wedge b\}c\,\{A\}}{\vdash \{A\}while\ b\ do\ c\,\{A \wedge not\ b\}}$$

Can be combined together with the rule of consequence

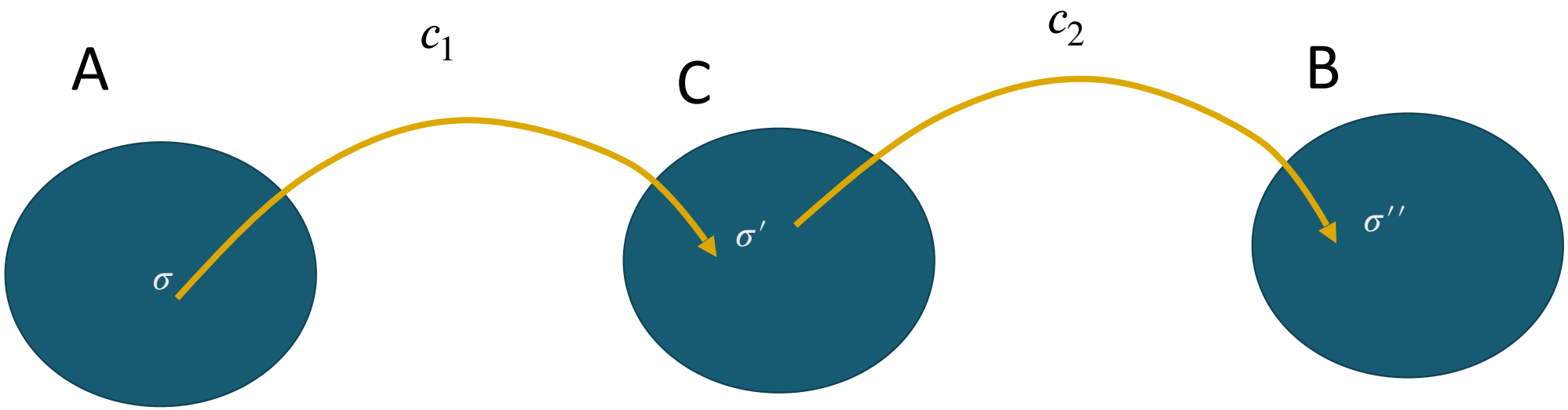
$$\frac{\vdash A' \Rightarrow A \vdash \{A\}c\,\{B\} \vdash B \Rightarrow B'}{\vdash \{A'\}c\,\{B'\}}$$

Visualizing the rules



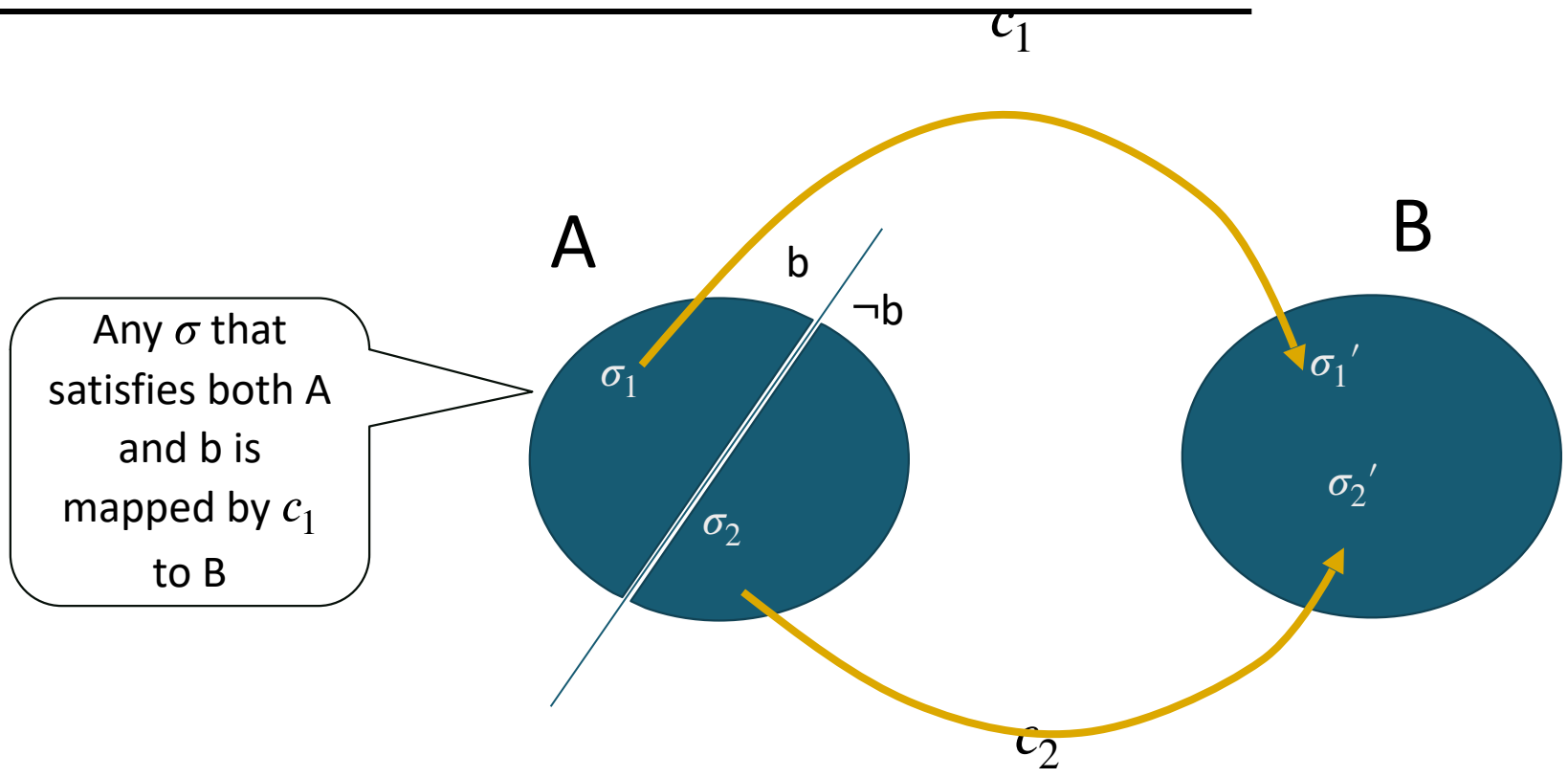
$$\{A\}\ cmd\ \{B\}$$

Visualizing the rules



$$\frac{\vdash \{A\}c_1\{C\} \quad \vdash \{C\}c_2\{B\}}{\vdash \{A\}c_1;c_2\{B\}}$$

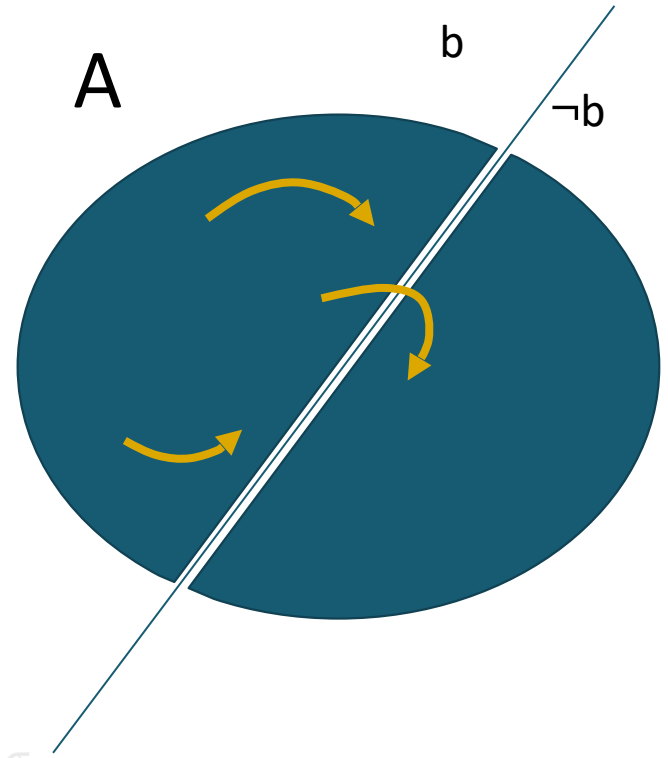
Visualizing the rules



$$\frac{\vdash \{A \wedge b\} c_1 \{B\} \qquad \vdash \{A \wedge not\ b\} c_2 \{B\}}{\vdash \{A\} if\ b\ then\ c_1\ else\ c_2 \{B\}}$$

Visualizing the rules

If c maps any point in A and b to another point in A, by the time iteration stops, we are guaranteed to be in $A \wedge \neg b$



$$\frac{\vdash \{A \wedge b\} c \{A\}}{\vdash \{A\} \mathit{while}\ b\ \mathit{do}\ c \{A \wedge \mathit{not}\ b\}}$$

Running Example

$\{x = xold \quad y = yold \; x > y\}$

```
if(x > y){
  t = x – y;
  while(t > 0){
    x = x – 1;
    y = y + 1;
    t = t – 1;
  }
}
{
```

Example

$$\left\{ \begin{array}{c} x = x_{old} \\ y = y_{old} \\ x > y \end{array} \right\}$$

```
if(x > y)
  t = x – y;
  while(t > 0)
    x = x – 1;
    y = y + 1;
    t = t – 1;
```

$$\left\{ \begin{array}{c} y = x_{old} \\ x = y_{old} \end{array} \right\}$$

Example

$$\frac{\vdash \{A \wedge b\} c_1 \{B\} \qquad \vdash \{A \wedge not\ b\} c_2 \{B\}}{\vdash \{A\} if\ b\ then\ c_1 else\ c_2 \{B\}}$$

$$\left\{ \begin{array}{l} x = x_{old} \\ y = y_{old} \quad x > y \\ x > y \end{array} \right\}$$

```
t = x – y;
while(t > 0)
  x = x – 1;
  y = y + 1;
  t = t – 1;
```

$$\left\{ \begin{array}{l} y = x_{old} \\ x = y_{old} \end{array} \right\}$$

$$\left\{ \begin{array}{l} x = x_{old} \\ y = y_{old} \quad not\ x > y \\ x > y \end{array} \right\}$$

skip

$$\left\{ \begin{array}{l} y = x_{old} \\ x = y_{old} \end{array} \right\}$$

$$\left\{ \begin{array}{l} x = x_{old} \\ y = y_{old} \\ x > y \end{array} \right\}$$

```
if(x > y)
  t = x – y;
  while(t > 0)
    x = x – 1;
    y = y + 1;
    t = t – 1;
```

$$\left\{ \begin{array}{l} y = x_{old} \\ x = y_{old} \end{array} \right\}$$

Example

$$\frac{\vdash A' \Rightarrow A \vdash \{A\}c\{B\} \vdash B \Rightarrow B'}{\vdash \{A'\}c\{B'\}}$$

$$\frac{\{False\} \Rightarrow \left\{\begin{array}{l} y = x_{old} \\ x = y_{old} \end{array}\right\} \quad \left\{\begin{array}{l} y = x_{old} \\ x = y_{old} \end{array}\right\} \text{ skip } \left\{\begin{array}{l} y = x_{old} \\ x = y_{old} \end{array}\right\}}{\left\{\begin{array}{l} x = x_{old} \\ y = y_{old} \\ x > y \end{array}\right\} \quad \begin{array}{l} \text{t = x – y;} \\ \text{while(t > 0)} \\ \quad \text{x = x – 1;} \\ \quad \text{y = y + 1;} \\ \quad \text{t = t – 1;} \end{array} \quad \left\{\begin{array}{l} y = x_{old} \\ x = y_{old} \end{array}\right\}}$$

$$\frac{\left\{\begin{array}{l} x = x_{old} \\ y = y_{old} \\ x > y \end{array}\right\} \quad \text{if(x > y)} \quad \left\{\begin{array}{l} y = x_{old} \\ x = y_{old} \end{array}\right\}}{\left\{\begin{array}{l} x = x_{old} \\ y = y_{old} \\ x > y \end{array}\right\} \quad \begin{array}{l} \text{t = x – y;} \\ \text{while(t > 0)} \\ \quad \text{x = x – 1;} \\ \quad \text{y = y + 1;} \\ \quad \text{t = t – 1;} \end{array} \quad \left\{\begin{array}{l} y = x_{old} \\ x = y_{old} \end{array}\right\}}$$

$$\left\{\begin{array}{l} x = x_{old} \\ y = y_{old} \\ x > y \end{array}\right\} \quad \text{False} \quad x > y$$

Example

$$\vdash \{A\} skip \{A\}$$

$$\{False\} \Rightarrow \left\{\begin{array}{l} y = x_{old} \\ x = y_{old} \end{array}\right\} \quad \left\{\begin{array}{l} y = x_{old} \\ x = y_{old} \end{array}\right\} \text{ skip } \left\{\begin{array}{l} y = x_{old} \\ x = y_{old} \end{array}\right\}$$

$$\left\{\begin{array}{l} x = x_{old} \\ y = y_{old} \\ x > y \end{array}\right\} \quad x > y \quad \left\{\begin{array}{l} x = x_{old} \\ y = y \\ x > y \end{array}\right\} \text{ False } \quad \text{skip} \quad \left\{\begin{array}{l} y = x_{old} \\ x = y_{old} \end{array}\right\}$$

$$\left\{\begin{array}{l} x = x_{old} \\ y = y_{old} \\ x > y \end{array}\right\}$$

```
t = x – y;
while(t > 0)
  x = x – 1;
  y = y + 1;
  t = t – 1;
```

$$\left\{\begin{array}{l} y = x_{old} \\ x = y_{old} \end{array}\right\}$$

$$\left\{\begin{array}{l} x = x_{old} \\ y = y_{old} \\ x > y \end{array}\right\}$$

```
if(x > y)
  t = x – y;
  while(t > 0)
    x = x – 1;
    y = y + 1;
    t = t – 1;
```

$$\left\{\begin{array}{l} y = x_{old} \\ x = y_{old} \end{array}\right\}$$

Example

$$\left\{\begin{array}{l} x = x_{old} \\ y = y_{old} \\ x > y \end{array}\right\} \quad x > y$$

```
t = x – y;
while(t > 0)
  x = x – 1;
  y = y + 1;
  t = t – 1;
```

$$\left\{\begin{array}{l} y = x_{old} \\ x = y_{old} \end{array}\right\}$$

$$\{False\} \Rightarrow \left\{\begin{array}{l} y = x_{old} \\ x = y_{old} \end{array}\right\} \qquad \left\{\begin{array}{l} y = x_{old} \\ x = y_{old} \end{array}\right\} \text{ skip } \left\{\begin{array}{l} y = x_{old} \\ x = y_{old} \end{array}\right\}$$

$$\{False\} \qquad \text{skip} \qquad \left\{\begin{array}{l} y = x_{old} \\ x = y_{old} \end{array}\right\}$$

$$\left\{\begin{array}{l} x = x_{old} \\ y = y_{old} \\ x > y \end{array}\right\}$$

```
if(x > y)
  t = x – y;
  while(t > 0)
    x = x – 1;
    y = y + 1;
    t = t – 1;
```

$$\left\{\begin{array}{l} y = x_{old} \\ x = y_{old} \end{array}\right\}$$

Example

$$\frac{\vdash \{A\}c_1\ \{C\} \quad \vdash \{C\}c_2\ \{B\}}{\vdash \{A\}c_1; c_2\ \{B\}}$$

$$\left\{\begin{array}{l} x = x_{old} \\ y = y_{old} \\ x > y \end{array}\right\}$$ t = x – y; $$\left\{\begin{array}{l} x = x_{old} \\ y = y_{old} \\ x > y \\ t = x_{old} - y_{old} \end{array}\right\}$$ $x > y$ $$\left\{\begin{array}{l} x = x_{old} \\ y = y_{old} \\ x > y \\ t = x_{old} - y_{old} \end{array}\right\}$$ $x > y$

while(t > 0)
x = x – 1;
y = y + 1;
t = t – 1;

$$\left\{\begin{array}{l} y = x_{old} \\ x = y_{old} \end{array}\right\}$$

$$\left\{\begin{array}{l} x = x_{old} \\ y = y_{old} \\ x > y \end{array}\right\}$$ $x > y$

t = x – y;
while(t > 0)
x = x – 1;
y = y + 1;
t = t – 1;

$$\left\{\begin{array}{l} y = x_{old} \\ x = y_{old} \end{array}\right\}$$

Example

$$\frac{\vdash A' \Rightarrow A \vdash \{A\}c\{B\} \vdash B \Rightarrow B'}{\vdash \{A'\}c\{B'\}}$$

$$\begin{pmatrix} x = x_{old} \\ y = y_{old} \quad x > y \\ x > y \end{pmatrix} \Rightarrow \begin{pmatrix} x = x_{old} \\ y = y_{old} \\ x > y \qquad\qquad x > y \\ x - y = x_{old} - y_{old} \end{pmatrix}$$

$$\left\{\begin{matrix} x = x_{old} \\ y = y_{old} \\ x > y \qquad\qquad x > y \\ x - y = x_{old} - y_{old} \end{matrix}\right\} \quad \text{t = x – y;} \quad \left\{\begin{matrix} x = x_{old} \\ y = y_{old} \\ x > y \qquad\qquad x > y \\ t = x_{old} - y_{old} \end{matrix}\right\}$$

$$\left\{\begin{matrix} x = x_{old} \\ y = y_{old} \quad x > y \\ x > y \end{matrix}\right\} \text{ t = x – y; } \left\{\begin{matrix} x = x_{old} \\ y = y_{old} \\ x > y \qquad\qquad x > y \\ t = x_{old} - y_{old} \end{matrix}\right\} \left\{\begin{matrix} x = x_{old} \\ y = y_{old} \\ x > y \qquad\qquad x > y \\ t = x_{old} - y_{old} \end{matrix}\right\}$$

```
while(t > 0)
  x = x – 1;
  y = y + 1;
  t = t – 1;
```

$$\left\{\begin{matrix} y = x_{old} \\ x = y_{old} \end{matrix}\right\}$$

$$\left\{\begin{matrix} x = x_{old} \\ y = y_{old} \quad x > y \\ x > y \end{matrix}\right\}$$

```
t = x – y;
  while(t > 0)
    x = x – 1;
    y = y + 1;
    t = t – 1;
```

$$\left\{\begin{matrix} y = x_{old} \\ x = y_{old} \end{matrix}\right\}$$

Example

$$\frac{\vdash A' \Rightarrow A \vdash \{A\}c\{B\} \vdash B \Rightarrow B'}{\vdash \{A'\}c\{B'\}}$$

$$\begin{pmatrix} x = x_{old} \\ y = y_{old} \quad x > y \\ x > y \end{pmatrix} \Rightarrow \begin{pmatrix} x = x_{old} \\ y = y_{old} \\ x > y \qquad\qquad x > y \\ x - y = x_{old} - y_{old} \end{pmatrix}$$

$$\left\{\begin{matrix} x = x_{old} \\ y = y_{old} \\ x > y \qquad\qquad x > y \\ x - y = x_{old} - y_{old} \end{matrix}\right\} \quad \text{t = x – y;} \quad \left\{\begin{matrix} x = x_{old} \\ y = y_{old} \\ x > y \qquad\qquad x > y \\ t = x_{old} - y_{old} \end{matrix}\right\}$$

$$\left\{\begin{matrix} x = x_{old} \\ y = y_{old} \quad x > y \\ x > y \end{matrix}\right\} \text{ t = x – y; } \left\{\begin{matrix} x = x_{old} \\ y = y_{old} \\ x > y \qquad\qquad x > y \\ t = x_{old} - y_{old} \end{matrix}\right\}$$

$$\left\{\begin{matrix} x = x_{old} \\ y = y_{old} \\ x > y \qquad\qquad x > y \\ t = x_{old} - y_{old} \end{matrix}\right\}$$

```
while(t > 0)
  x = x – 1;
  y = y + 1;
  t = t – 1;
```

$$\left\{\begin{matrix} y = x_{old} \\ x = y_{old} \end{matrix}\right\}$$

$$\left\{\begin{matrix} x = x_{old} \\ y = y_{old} \quad x > y \\ x > y \end{matrix}\right\}$$

```
t = x – y;
  while(t > 0)
    x = x – 1;
    y = y + 1;
    t = t – 1;
```

$$\left\{\begin{matrix} y = x_{old} \\ x = y_{old} \end{matrix}\right\}$$

Example

$$\frac{\vdash \{A \wedge b\} c \{A\}}{\vdash \{A\} \mathit{while}\ b\ \mathit{do}\ c \{A \wedge \mathit{not}\ b\}}$$

$$\frac{\vdash A' \Rightarrow A \vdash \{A\} c \{B\} \vdash B \Rightarrow B'}{\vdash \{A'\} c \{B'\}}$$

$$\frac{\{A \wedge t > 0\}\ \begin{array}{l}\text{x = x – 1;}\\ \text{y = y + 1;}\\ \text{t = t – 1;}\end{array}\ \{A\}}{\begin{pmatrix} x = x_{old} \\ y = y_{old} \\ x > y \\ t = x_{old} - y_{old} \end{pmatrix} \quad x > y \Rightarrow A \qquad \{A\}\ \begin{array}{l}\text{while(t > 0)}\\ \text{x = x – 1;}\\ \text{y = y + 1;}\\ \text{t = t – 1;}\end{array}\ \{A \wedge \ \mathit{not}\ t > 0\} \qquad A \wedge \mathit{not}\ t > 0 \Rightarrow \begin{pmatrix} y = x_{old} \\ x = y_{old} \end{pmatrix}}$$

$$\left\{ \begin{array}{l} x = x_{old} \\ y = y_{old} \\ x > y \\ t = x_{old} - y_{old} \end{array} \quad x > y \right\}\ \begin{array}{l}\text{while(t > 0)}\\ \text{x = x – 1;}\\ \text{y = y + 1;}\\ \text{t = t – 1;}\end{array}\ \left\{ \begin{array}{l} y = x_{old} \\ x = y_{old} \end{array} \right\}$$

Loop Invariant

{Pre} while(p) c {Post}

Need to find invariant A such that

- $Pre \Rightarrow A$
- $A \wedge \neg\, p \Rightarrow Post$
- $\{A \wedge p\} c \{A\}$

Example

$$\{A \wedge t > 0\} \quad \begin{array}{l} \text{x = x – 1;} \\ \text{y = y + 1;} \\ \text{t = t – 1;} \end{array} \quad \{A\}$$

$$\begin{pmatrix} x = x_{old} \\ y = y_{old} \\ x > y \\ t = x_{old} - y_{old} \end{pmatrix} \quad x > y \Rightarrow A$$

$$A \wedge not\ t > 0 \Rightarrow \begin{pmatrix} y = x_{old} \\ x = y_{old} \end{pmatrix}$$

Soundness and Completeness

What does it mean for our deduction rules to be sound?

- You will never be able to prove anything that is not true
- truth is defined in terms of our original definition of {A} c {B}

$$\forall\sigma\forall\sigma'(\sigma \vDash A \wedge \langle c,\sigma\rangle \rightarrow \sigma') \Rightarrow \sigma' \vDash B$$

- we can prove this, but it's tricky

What does it mean for them to be complete?

- If a statement is true, we should be able to prove it via deduction

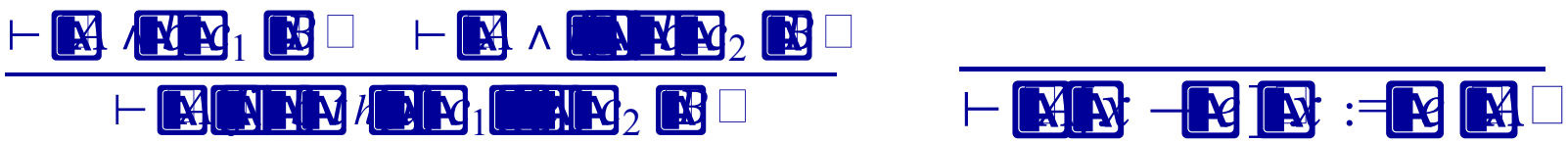
So are they complete?

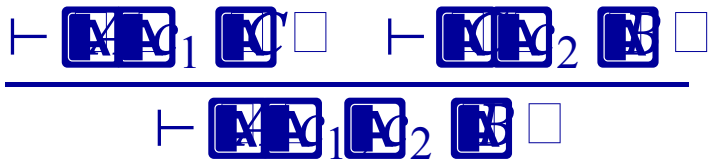
- yes and no
  - They are complete relative to the logic
  - but there are no complete and consistent logics for elementary arithmetic (Gödel)

From partial to total correctness

Total correctness judgment

- $\vdash [A]\ c\ [B]$
- Just like before, but must also prove termination





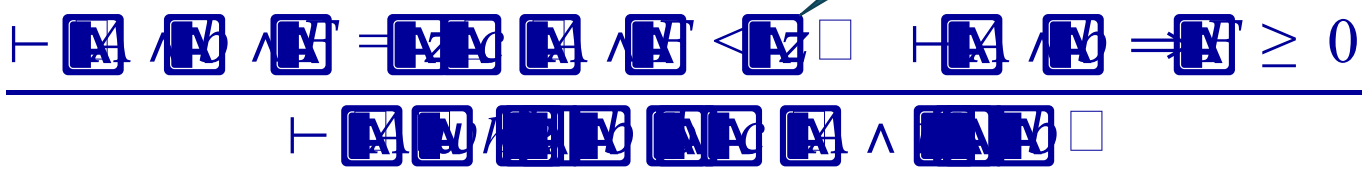
What about loops

Rank function

Function F of the state that

- a) Maps state to an integer
- b) Decreases with every iteration of the loop
- c) Is guaranteed to stay greater than zero
- Also called variant function

z is just a fresh variable not used elsewhere in the expressions



Example

Can we prove this?

```
[ x=x0 and y=y0 and x > y ]
if(x > y){
  t = x – y;
  while(t > 0){
    x = x – 1;
    y = y + 1;
    t = t – 1;
  }
}
[y=x0 and x=y0 ]
```

Example

$$\frac{\vdash [A_1 \wedge A_2 \wedge A_3 \Rightarrow A_4 A_5][A_1 \wedge A_3 \Rightarrow A_2 < A_5] \quad \vdash [A_1 \wedge A_2 \Rightarrow A_3 \geq 0]}{\vdash [A_1 A_2 \wedge A_1 A_2 A_3 \; A_1 A_2 A_3 \; A_1 \wedge A_1 A_2]}$$

$$x = y_{old} + t$$
$$A \equiv \wedge \; y = x_{old} - t \quad F \equiv t$$
$$\wedge \quad t \geq 0$$

$[A \wedge t > 0 \wedge t = z]$

```
x = x – 1;
y = y + 1;
t = t – 1;
```

$[A \wedge (t < z)]$

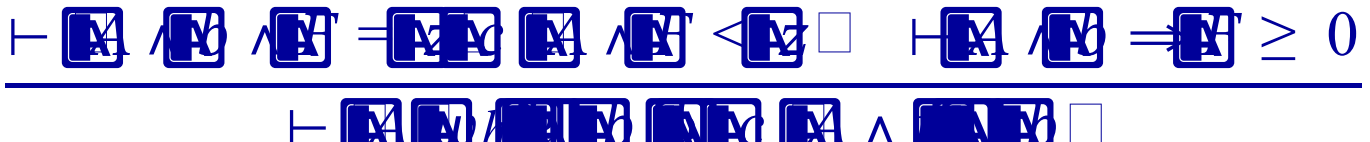
$A \wedge t > 0 \Rightarrow t \geq 0$ ✔

$[A]$

```
while(t > 0)
  x = x – 1;
  y = y + 1;
  t = t – 1;
```

$[A \wedge \quad not\ t > 0]$

Example



$$\left[\begin{array}{l} x = y_{old} + t \\ \wedge\ y = x_{old} + t \quad \wedge t = z \\ \wedge\ \ t \geq 0 \end{array}\right] \texttt{x = x – 1;} \texttt{y = y + 1;} \left[\begin{array}{l} x = y_{old} + (t-1) \\ \wedge\ y = x_{old} - (t-1) \quad \wedge (t = z) \\ \wedge\ \ (t-1) \geq 0 \end{array}\right]$$

$$\left[\begin{array}{l} x = y_{old} + (t-1) \\ \wedge\ y = x_{old} - (t-1) \quad \wedge (t = z) \\ \wedge\ \ (t-1) \geq 0 \end{array}\right] \texttt{t = t – 1;} \left[\begin{array}{l} x = y_{old} + t \\ \wedge\ y = x_{old} - t \quad \wedge (t < z) \\ \wedge\ \ t \geq 0 \end{array}\right]$$

$$\left[A \wedge t > 0 \wedge t = z\right]$$

```
x = x – 1;
y = y + 1;
t = t – 1;
```

$$\left[A \wedge (t < z)\right]$$

$$A \wedge t > 0 \Rightarrow t \geq 0$$

$$\left[A\right]$$

```
while(t > 0)
  x = x – 1;
  y = y + 1;
  t = t – 1;
```

$$[A \wedge \quad not\ t > 0]$$

Example

$$\begin{pmatrix} x = y_{old} + (t-1) \\ \wedge\ y = x_{old} - (t-1) \quad \wedge (t = z) \\ \wedge\ (t-1) \geq 0 \end{pmatrix} \Rightarrow \begin{pmatrix} x = y_{old} + (t-1) \\ \wedge\ y = x_{old} - (t-1) \quad \wedge (t-1 < z) \\ \wedge\ (t-1) \geq 0 \end{pmatrix}$$

$$\left[\begin{array}{l} x = y_{old} + (t-1) \\ \wedge\ y = x_{old} - (t-1) \quad \wedge (t-1 < z) \\ \wedge\ (t-1) \geq 0 \end{array}\right] \text{ t = t – 1; } \left[\begin{array}{l} x = y_{old} + t \\ \wedge\ y = x_{old} - t \quad \wedge (t < z) \\ \wedge\ t \geq 0 \end{array}\right]$$

…

$$\left[\begin{array}{l} x = y_{old} + (t-1) \\ \wedge\ y = x_{old} - (t-1) \quad \wedge (t = z) \\ \wedge\ (t-1) \geq 0 \end{array}\right] \text{ t = t – 1; } \left[\begin{array}{l} x = y_{old} + t \\ \wedge\ y = x_{old} - t \quad \wedge (t < z) \\ \wedge\ t \geq 0 \end{array}\right]$$

$[A \wedge t > 0 \wedge t = z]$

x = x – 1;
y = y + 1;
t = t – 1;

$[A \wedge (t < z)]$

$A \wedge t > 0 \Rightarrow t \geq 0$

$[A]$

while(t > 0)
  x = x – 1;
  y = y + 1;
  t = t – 1;

$[A \wedge \quad not\ t > 0]$

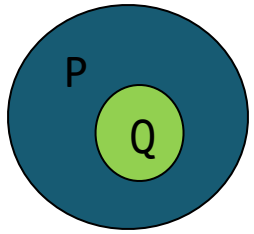
Weakest Preconditions

$$\mathrm{P} = wpc(c,\ A)$$

Command

Predicate

Weakest predicate P such that $\{P\}\ c\ \{A\}$



- P weaker than Q iff $Q \Rightarrow P$

$$wpc(skip,\ Q) = Q$$

$$wpc(x := e,\ Q) = Q[x \rightarrow e]$$

$$wpc(C_1; C_2\ ,\ Q) = wpc(C_1,\ wpc(C2, Q))$$

$$wpc(if\ B\ then\ C_1\ else\ C_2,\ Q) = (B \wedge wpc(C_1,\ Q)) \vee (\neg B \wedge wpc(C_2,\ Q))$$

Weakest Precondition

While-loop is tricky

- Let W $= wpc(while\ e\ do\ c,\ B)$
- then,

$$W = e \Rightarrow wpc(c,\ W) \ \wedge \ \ \neg e \Rightarrow B$$

Verification Condition

Stronger than the weakest precondition

Can be computed by using an invariant

$$VC\left(while_I\ e\ do\ c,\ B\right) =$$

$$I \wedge \quad \forall x_1,\ \ldots x_n\ I \Rightarrow \left(e \Rightarrow VC(c, I) \wedge \neg e \Rightarrow B\right)$$

- Where x_i are variables modified in c.